

Learn ESP32 for Home Automation

Lesson 7: Integrating Sensors (Temperature & Motion)

Integrating Sensors (Temperature & Motion) - Study Notes

Key Concepts

1. **DHT22 Temperature & Humidity Sensor** – how it works, wiring, and reading data.
2. **PIR (Passive Infrared) Motion Sensor** – detection principle, wiring, and signal handling.
3. **MQTT vs. HTTP** – lightweight messaging protocols for sending sensor data to a server or cloud.
4. **Microcontroller code flow** – initializing sensors, reading values, formatting payloads, and publishing.
5. **Basic error handling & power considerations** – ensuring reliable operation in a beginner project.

Summary

In this lesson you learned how to connect two of the most common low cost sensors to a microcontroller (e.g., ESP8266, ESP32, Arduino Uno) and transmit the measurements to a remote endpoint. The DHT22 provides temperature and relative humidity values via a single wire digital interface; the PIR sensor outputs a simple HIGH/LOW signal when it detects motion. After wiring the sensors and installing the appropriate libraries, the microcontroller reads the sensor data at regular intervals, builds a JSON payload, and sends it either to an MQTT broker (publish/subscribe model) or to an HTTP endpoint (REST POST).

Both MQTT and HTTP have their own strengths: MQTT is ideal for low bandwidth, real time telemetry and scales well with many devices, while HTTP is straightforward for quick testing or integration with web services that already expect REST calls. The lesson also covered basic troubleshooting—checking wiring, confirming sensor readings in the serial monitor, and handling connection failures gracefully.

Important Points

- **DHT22 wiring**: VCC !' 3.3 V (or 5 V), GND !' GND, DATA !' digital pin with a **4.7 kΩ** pull up resistor to VCC.
- **PIR wiring**: VCC !' 5 V (or 3.3 V depending on module), GND !' GND, OUT !' digital input pin. Some modules have a **potentiometer** to adjust sensitivity and a **delay potentiometer** for the “off” time.
- **Libraries**:
 - Arduino: ``DHT.h`` (or ``Adafruit_Sensor`` + ``DHT_U.h``)
 - ESP8266/ESP32: ``PubSubClient`` for MQTT, ``ESP8266HTTPClient`` / ``HTTPClient`` for HTTP.
- **MQTT basics**:
 - Broker URL, port (usually 1883 for non TLS).
 - Topic hierarchy, e.g., ``home/livingroom/temperature``.
 - QoS levels 0-2; beginners usually start with QoS /0.
- **HTTP basics**:

- POST to `http://yourserver/api/sensor` with `Content-Type: application/json`.
- Use `WiFiClientSecure` for HTTPS (port /443).

- **Payload example** (JSON):

```
```json
{
 "device_id": "sensor01",
 "temperature": 23.5,
 "humidity": 55,
 "motion": true,
 "timestamp": 1714521600
}
```
```

- **Timing**:

- DHT22 can be read every **2 /seconds** (max 0.5 /Hz).
- PIR can be polled or use an interrupt; debounce with a short software delay ("H200 /ms).

- **Error handling**:

- Check return value of `dht.readTemperature()` / `readHumidity()`.
- Re connect Wi Fi and MQTT if `client.connected()` is false.

- **Power**:

- DHT22 draws < /2 /mA when measuring; PIR draws ~**20 /mA** when active.
- Put the microcontroller in deep sleep between readings if battery powered.

Examples

Example 1 – Sending DHT22 & PIR data via MQTT (ESP8266)

```
```cpp
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>

#define DHTPIN D4 // GPIO2 on ESP 01, change as needed
#define DHTTYPE DHT22
#define PIRPIN D5 // GPIO14

const char* ssid = "YourWiFi";
const char* password = "YourPass";
const char* mqtt_server = "broker.hivemq.com";

WiFiClient espClient;
PubSubClient client(espClient);
DHT dht(DHTPIN, DHTTYPE);

void setup() {
 Serial.begin(115200);
 pinMode(PIRPIN, INPUT);
 dht.begin();
}
```

```

WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
 delay(500);
 Serial.print(".");
}
Serial.println("\nWiFi connected");
client.setServer(mqtt_server, 1883);
reconnect();
}

void reconnect() {
 while (!client.connected()) {
 Serial.print("Attempting MQTT connect...");
 if (client.connect("esp8266_sensor01")) {
 Serial.println("connected");
 } else {
 Serial.print("failed, rc=");
 Serial.print(client.state());
 delay(2000);
 }
 }
}

void loop() {
 if (!client.connected()) reconnect();
 client.loop();
 float t = dht.readTemperature();
 float h = dht.readHumidity();
 bool motion = digitalRead(PIRPIN) == HIGH;
 if (isnan(t) || isnan(h)) {
 Serial.println("Failed to read DHT22");
 return;
 }
 // Build JSON payload
 String payload = "{\"device\":\"sensor01\",\"temp\":";
 payload += String(t,1);
 payload += "\",\"hum\":";
 payload += String(h,1);
 payload += "\",\"motion\":";
 payload += motion ? "true" : "false";
 payload += "}";
 // Publish
 client.publish("home/livingroom/sensor", payload.c_str());
 Serial.println("Published: " + payload);
}

```

```

 delay(2000); // DHT22 min interval
}
...

```

\*Explanation\*: The sketch connects to Wi Fi, then to a public MQTT broker. Every 2 /seconds it reads temperature, humidity, and motion, builds a JSON string, and publishes to `home/livingroom/sensor`. Errors are printed to the serial monitor.

---

## Example 2 – Posting the same data to an HTTP endpoint (ESP32)

```

```cpp
#include <WiFi.h>
#include <HTTPClient.h>
#include <DHT.h>
#define DHTPIN 4      // GPIO4
#define DHTTYPE DHT22
#define PIRPIN 5      // GPIO5
const char* ssid     = "YourWiFi";
const char* password = "YourPass";
const char* serverURL = "http://example.com/api/sensor";
DHT dht(DHTPIN, DHTTYPE);

void setup() {
    Serial.begin(115200);
    pinMode(PIRPIN, INPUT);
    dht.begin();
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nWiFi connected");
}

void loop() {
    float t = dht.readTemperature();
    float h = dht.readHumidity();
    bool motion = digitalRead(PIRPIN) == HIGH;
    if (isnan(t) || isnan(h)) {
        Serial.println("DHT22 read error");
        delay(2000);
        return;
    }
    // JSON payload
    String json = "{\"device\":\"sensor01\", \"temperature\":";
    json += String(t,1);

```

```

json += "\",\"humidity\":";
json += String(h,1);
json += "\",\"motion\":";
json += motion ? "true" : "false";
json += "}";
if (WiFi.status() == WL_CONNECTED) {
  HTTPClient http;
  http.begin(serverURL);
  http.addHeader("Content-Type", "application/json");
  int httpResponseCode = http.POST(json);
  if (httpResponseCode > 0) {
    Serial.printf("POST sent, response %d\n", httpResponseCode);
  } else {
    Serial.printf("POST failed, error: %s\n", http.errorToString(httpResponseCode).c_str());
  }
  http.end();
} else {
  Serial.println("WiFi disconnected");
}
delay(2000);
}
...

```

***Explanation*:** This version uses the `HTTPClient` library to POST the same JSON payload to a REST endpoint. It demonstrates how to set headers, handle the response code, and reconnect Wi-Fi if needed.

Quick Review

1. ****What pull-up resistor value is recommended for the DHT22 data line, and why?***
2. ****Name two key differences between MQTT and HTTP for sensor telemetry.*****
3. ****If the DHT22 returns `nan`, what are three possible causes?*****
4. ****How can you reduce power consumption when using a PIR sensor on a battery-powered board?*****
5. ****Write a one-line JSON snippet that represents a motion-detected event (device id "sensor02").*****

Further Reading

- ****MQTT Advanced Topics**** – retained messages, Last Will & Testament, TLS security.
- ****Deep Sleep on ESP8266/ESP32**** – waking on timer vs. external interrupt (useful for PIR).
- ****Sensor Calibration**** – improving DHT22 accuracy, handling humidity drift.
- ****Home Automation Platforms**** – integrating MQTT topics with Home Assistant or OpenHAB.
- ****Secure HTTP (HTTPS) on microcontrollers**** – using `WiFiClientSecure` and certificate pinning.

Happy tinkering! Ø=Þ€

